

VIDEO NOTES · PART 8

Memory in LangChain

Five ways to manage conversation history

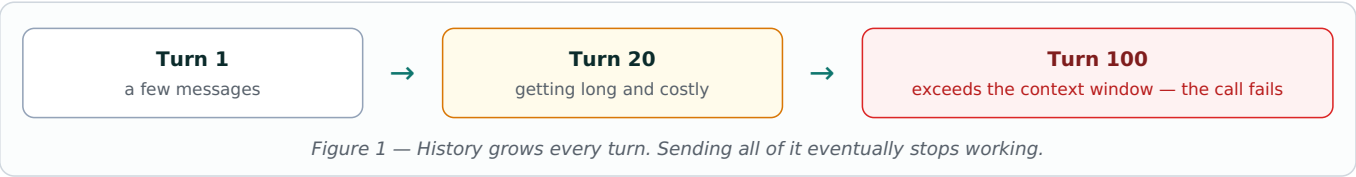
What this document is

A plain-English guide to the Memory component — why chatbots forget, and the five strategies for fixing it.

1 Why Memory Is Needed

An LLM call is **stateless**. Each request stands alone, and the model has no recollection of anything said before it. On its own, a chatbot forgets your name the moment you finish typing it.

The fix sounds simple: send the earlier messages along with the new one. And it is simple — until the conversation gets long.

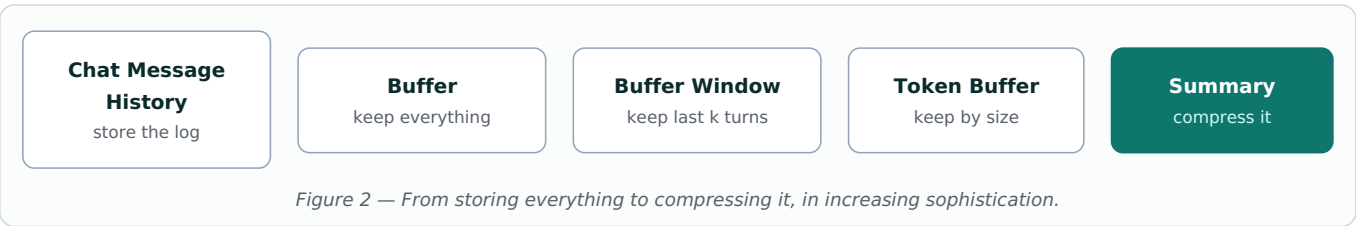


Two hard limits push back:

Constraint	What happens as history grows
The context window	Every model has a maximum amount of text it can accept in one call. Exceed it and the request fails outright.
Cost and speed	You pay per token, on every turn. Resending the whole history each time means the same early messages are charged for again and again, and each call gets slower.

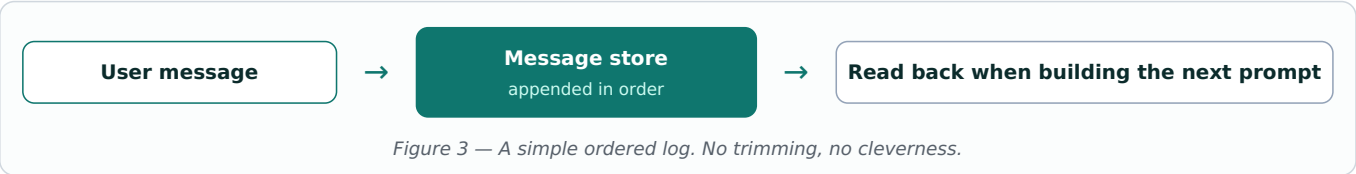
What memory actually is: a strategy for deciding *which parts* of the conversation to send with the next request. Every option below is a different answer to that single question.

2 The Five Options



3 Chat Message History

This is the foundation the others build on. It is a container that holds the messages of a conversation in order — you add each message as it happens, and read them all back when you need them.



By itself it does not solve the growth problem — it just stores. Its value is that it separates *where messages live* from *how much of them you use*.

Why the separation matters: the store can live in a database rather than in process memory. That is what lets a conversation survive a server restart, and lets a user pick up on their phone where they left off on a laptop.

This is also where the **message placeholder** from Part 6 comes in. The history goes into the reserved gap inside your chat template.

4 Conversation Buffer Memory

The simplest strategy: send everything. Every message, every turn, exactly as it happened.

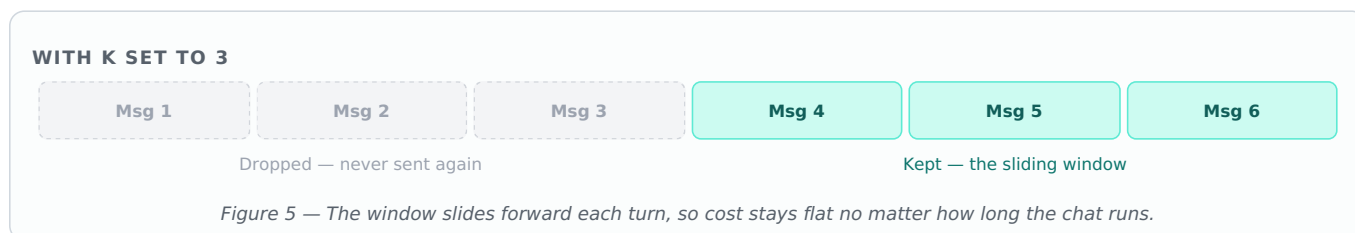


Strengths	Weaknesses
Nothing is ever lost. The model has complete context, so answers referring to something said much earlier work perfectly.	Cost and length grow without limit. Fine for a ten-turn exchange; it will eventually break a long one.

Use it when conversations are naturally short and bounded — a form-filling flow, a quick support query, a demo. Reach for something else the moment sessions can run long.

5 Conversation Buffer Window Memory

The first real limit. Keep only the most recent **k** exchanges and discard anything older.



The great advantage is predictability: the amount you send stops growing. Turn 500 costs the same as turn 5.

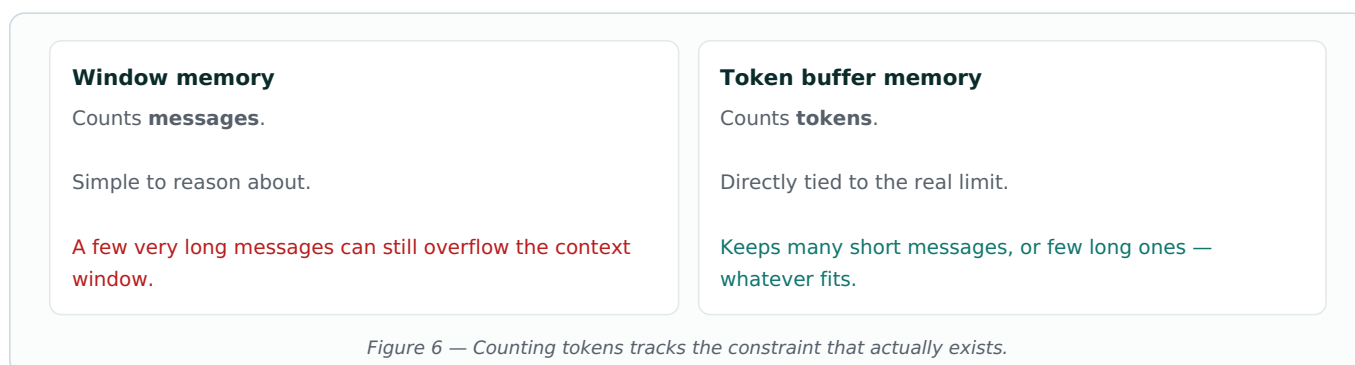
The trade-off is real, and abrupt. Once a message falls out of the window it is gone completely. If a user gave their order number at turn 2 and refers to "that order" at turn 20, the bot no longer knows what they mean.

Choosing **k** is a judgement call: large enough to cover how far back users typically refer, small enough to stay affordable.

6 Conversation Token Buffer Memory

Same sliding idea, but measured in **tokens** rather than message count.

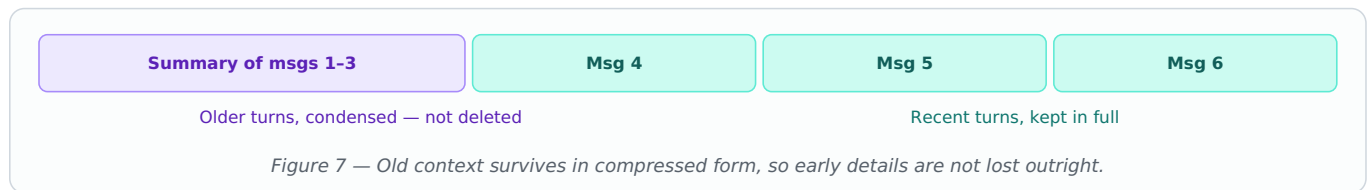
This matters because messages vary enormously in size. Three messages might be a hundred tokens or five thousand, depending on whether someone pasted a document into the chat.



Why this is usually the better default: the context window is measured in tokens, so a token-based limit maps onto it exactly. A message-count limit is only ever an approximation of the thing you actually care about.

7 Conversation Summary Memory

The other four strategies all keep messages verbatim and differ only in how many. This one takes a different approach: instead of dropping old messages, it uses an LLM to **compress** them into a running summary.



The order number given at turn 2 can still appear in the summary at turn 50. That is the thing window memory cannot do.

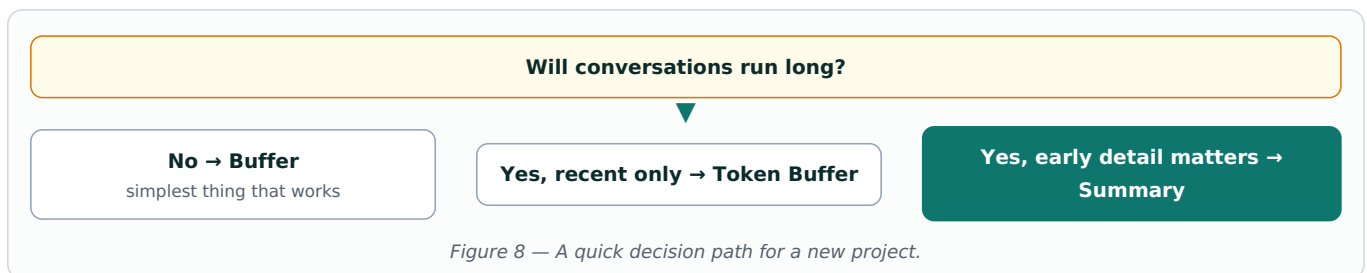
What it costs

Consideration	What to keep in mind
An extra model call	Summarising is itself an LLM call. You are trading tokens spent on history for tokens spent on compression.
Detail is lost	A summary keeps the gist. Exact figures, names and wording may not survive, and you cannot know in advance which detail will matter later.
Errors compound	Each new summary is built partly from the previous one. A small distortion early can persist and drift as the conversation continues.
Added latency	The summarisation step sits inside your response time.

A practical middle ground: keep recent turns verbatim and summarise only what falls out of that window — exactly what Figure 7 shows. Recent detail stays exact, older context stays available in compressed form.

8 Choosing a Strategy

Strategy	Keeps	Cost as chat grows	Use when
Chat Message History	Everything, as storage	Storage only	You need history to persist across sessions
Buffer	Every message	Grows without limit	Conversations are short and bounded
Buffer Window	Last k messages	Flat	Only recent context matters
Token Buffer	As much as fits	Flat, capped precisely	Message lengths vary a lot — a good default
Summary	A summary plus recent turns	Flat, plus summarising calls	Long conversations where early details still matter



Start simple. Begin with buffer memory while building. Move to a token buffer when conversations lengthen, and add summarisation only if users genuinely refer back to things said long ago. Choosing the most sophisticated option first usually adds cost without adding value.

9 A Note on Sharing Your Work

The instructor returns to a theme from the opening lecture: post your learning journey publicly, on LinkedIn or elsewhere.

The reasoning is not self-promotion for its own sake. Writing an explanation forces you to find the gaps in your own understanding — you cannot summarise a memory strategy clearly if you do not actually know when you would pick it. The public record is a by-product of that.

A low-effort version: after each lecture, post what the component does, one thing that surprised you, and one trade-off you had not considered. Three sentences. Consistency matters more than length.

Practice before the next lecture

- ☐ Build a chatbot with no memory and confirm it forgets your name.
- ☐ Add buffer memory and watch the same question now work.
- ☐ Switch to window memory with a small k, then ask about something from the first turn.
- ☐ Paste a long block of text into a chat using window memory and see the token count spike.
- ☐ Swap in token buffer memory and confirm the same conversation stays within budget.
- ☐ Run a long conversation with summary memory and read the summary it built.

The short version: memory is the strategy for deciding what history to send. Buffer keeps everything, window keeps

the last few, token buffer keeps as much as fits, and summary compresses the rest — each trading completeness against cost in a different way.